

Authentication of administrative documents using blockchain technology

Zilèdem Pierre Canisius HIEN
Department of Informatics
(DI)

University Joseph KI-ZERBO
(UJKZ)

OUAGADOUGOU, BURKINA FASO
hiencanisius@gmail.com

Ousmane BARRA
Department of Informatics
(DI)

University Joseph KI-ZERBO
(UJKZ)

OUAGADOUGOU, BURKINA FASO
ousmane.barra@ujkz.bf

Yaya TRAORE
Department of Informatics
(DI)

University Joseph KI-ZERBO
(UJKZ)

OUAGADOUGOU, BURKINA FASO
yaya.traore@tic.gov.bf

Sadouanouan MALO
Department of Informatics
(DI)

University NAZI BONI
(UNB)

BOBO-DIOULASSO, BURKINA
FASO
sadouanouan.malo@u-naziboni.bf

Abstract— Administrative documents have always played a central role not only in the functioning of the Administration but also in the relationship between the Administration and its users/customers. Indeed, they are used for traceability and serve as a basis for decision-making, communication and legal purposes. This means that all administrative documents must be reliable and of indisputable origin. However, it is increasingly common to encounter falsified digital administrative documents or those of dubious origin. This phenomenon creates a crisis of confidence between users/customers and the administration. Blockchain is a revolutionary technology can help secure administrative documents. In this paper, we propose a decentralized approach to authenticating administrative documents based on the non-repudiation, transparency and decentralized storage properties of blockchain technology. We combine the use of a smart contract, the SHA-256 hashing algorithm, digital signatures and electronic time stamping. Our process begins with the registration of administrative documents on the blockchain by an administrative entity. Thus, any user (or other administrative entity) wishing to verify the authenticity of an administrative document in their possession can also upload it to the Ethereum blockchain, which will (after research and processing) confirm or deny the authenticity of said document. This solution not only guarantees greater security and integrity for administrative documents, but also offers the ability to verify their authenticity in real time in a simple manner. Simulations show that it is potentially useful for administration.

Keywords— authentication, security, document, blockchain, cryptography, decentralized application

I. INTRODUCTION

Document authentication is a process whereby a computer system or a human being proves or certifies that a document is authentic. A document is considered authentic if it is the original, or a copy that conforms to the original after verification and validation by an authorised or competent person. However, this form of certification is not intended to prove the veracity of the document's content. Its purpose is to ensure that the signature and stamp visible on the document originate from an official authority and that the date of signature or issue, the name and position of the signatory are also legible.

The reliability of certain administrative documents is increasingly controversial, as it is now common to find falsified documents or documents of dubious origin in the public digital space, causing confusion among many citizens. This could be due in part to the development and misuse of multiple tools based, for example, on Artificial Intelligence (AI).

Indeed, the phenomenon of falsified documents contributes to tarnishing the image of their legitimate author, which could be the government, thus creating a crisis of confidence between the government and its users/customers. To address this phenomenon, authors and recipients of documents sometimes carry out authentication operations, at enormous cost in terms of time and money, which could ultimately reduce their returns.

Blockchain is a technological revolution that has been rapidly developing in recent years. Known for its non-repudiation, transparency and decentralized storage properties, it offers numerous advantages in many aspects of human life and is proving to be an effective means of improving government public services in particular. As a result, the use of this technology in the authentication process for administrative documents could bring greater ease, reliability and security.

In this paper we propose an approach to document authentication using blockchain technology. Indeed, are therefore interested in the use of blockchain technology, particularly its consensus, verification and validation protocols, to resolve the issue of authenticating administrative documents, in order to reduce the risk of fraud and falsification of these documents.

The rest of the paper is organized as follows: in Section 2 we present a state-of-the-art review of existing work. In Section 3 we present our methodology. In section 4 we present a simulation of the approach. We end with a conclusion and prospects in Section 5.

II. STATE OF THE ART

Blockchain technology, propelled by the advent in 2008 of Satoshi Nakamoto's bitcoin cryptocurrency [1], has been growing rapidly in recent decades. It is a digital technology

that allows information to be stored chronologically and transmitted in the form of blocks linked together securely (heavy use of cryptography) and without a central authority.

According to *Imran Bashir* [2], third-generation blockchains are applicable beyond the financial services industry in various general-purpose societal areas such as media, healthcare, government, etc. In addition to Bitcoin, there are several blockchains (Solana, Ethereum, Cardano, Avalanche, etc.), among which we have chosen Ethereum for this experiment.

A. Ethereum Blockchain

The public Ethereum blockchain, whose consensus mechanism switched from Proof of Work (PoW) to Proof of Stake (PoS) in September 2022 [3], underwent several changes after it was announced at the North American Bitcoin Conference in Miami in January 2014 and then launched on 30 July 2015 [4]. In the white paper published in 2013 [5], *Vitalik Buterin* stated that “Ethereum’s goal is to create another protocol for developing decentralized applications, offering a different set of trade-offs that we believe will be very useful for a wide range of decentralized applications. It will primarily focus on situations where rapid development, security for small, rarely used applications, and the ability for different applications to interact with each other in a highly efficient manner are important.”. In line with this dynamic, Ethereum 2.0 regularly introduces numerous developments, such as its May 7, 2025 update via the Pectra hard fork, which makes the network more secure, accessible, and user-friendly.

The ability to produce decentralized applications based on Ethereum – operating on a peer-to-peer network – relies heavily on the use of smart contracts. This makes Ethereum a Turing Complete blockchain [6], unlike the Bitcoin network. In certain use cases of Ethereum, such as ours, we must use two (02) accounts. The first is an **Externally Owned Account (EOA)** controlled by a cryptographic private key, with several hundred ETH and identifiable by an address (a 42-characters hexadecimal hash prefixed with 0x). The second is a **Contract Account (CA)** whose address is obtained after deployment (using ≈ 782016 Gas) of our smart contract.

Technically, a smart contract refers to a computer program implemented in a high-level programming language such as Solidity, Vyper, etc., whose code is compiled into bytecode and deployed on a specific version of the blockchain virtual machine – the EVM, in the case of Ethereum. It is designed to run autonomously when certain criteria are met. In addition to their advantages in terms of speed, accuracy, transparency and security, smart contracts are, according to [2], a revolutionary feature of blockchain because they allow for flexibility, empowerment and highly desirable control over the actions desired by users.

B. Document authentication and security

According to Article 4 of [7], the following are considered administrative documents: “documents produced or received, in the context of public service, by the State, local authorities and other public or private entities entrusted with such a mission;”. These include, for example, memos, decisions, instructions, circulars, directives, journals, deliberations, reports, minutes, sketches, plans, diagrams, notices, forecasts, official communiqués, certificates (of commencement, resumption, or termination of service, etc.), bulletins, decrees, orders, etc.

Given that a document refers to information stored on paper or electronic media, document authentication is a process whereby a computer system or human being proves or certifies that a document is authentic. In general, for authorized individuals – civil registrars, police officers, official authorities that issued the document, etc. – the process consists of ensuring that the signature and stamp visible on the document (original, certified copy of the original) originate from an official authority and that the date of signature or issue, the name and position of the signatory are also legible.

In all cases, authentication is a component of security, which itself is a set of measures to be taken and implemented to ensure traceability related to access and the protection of sensitive information (electronic or physical).

C. Existing methods of document authentication

There are several methods of authenticating digital documents.

- **Digital signature technology:** This secure method makes it possible to prove the identity of the author of an electronic document and guarantee the integrity of said document using public key cryptography. It thus guarantees non-repudiation, i.e. it is virtually impossible to challenge or deny the document. A digital signature must be authentic, unforgeable, non-reusable, unalterable and irrevocable [8].
- **Hashing or digital fingerprinting technique:** This technique involves applying a one-way algorithm (SHA-2, SHA-3, etc.) to the document to generate a unique digital fingerprint. This technique does not prove the identity of the author, but only the integrity of the document.
- **Electronic timestamping technique:** Most often based on digital signatures and hashing, this technique is used to prove that a document existed on a given date.
- **RFID and QR code techniques:** Radio frequency identification (RFID) and quick response codes (QR codes) are used in many sectors (commerce, administration, healthcare, etc.) to track certain goods or verify the authenticity of documents, particularly physical ones.
- **Blockchain technology:** *Isyak Meirobie et al.* [9] have set up a framework for authenticating electronic documents using Blockchain technology for the Indonesian government system. Go-Chain stores the hashed (in SHA256) signature (P-256 curve DSA) of the digital transcript of the document on a public blockchain. This transcript is also distributed to the public in json format. The author opted for the Proof of Authority (PoA) consensus protocol. *Ana BAKHOUM* [10] has also proposed a decentralized system for backing up and securing electronic school reports in the Ethereum blockchain via a smart contract. This aims to ensure the reliability, authenticity, transparency and security of electronic reports. Our solution innovates with the integration of electronic timestamping and the non-direct manipulation of cryptographic keys by the general public. It offers the possibility of verifying the authenticity of documents in real time, securely,

transparently, and without going through a central verification authority.

III. METHODOLOGY

Our authentication approach (Fig. 1) consists of two main phases that are ordered and complementary.

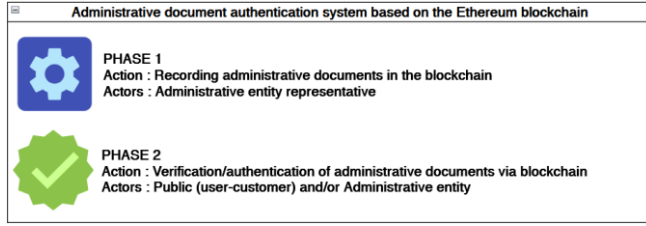


Fig. 1. Constitutive phases of the approach to authenticating administrative documents using the Ethereum blockchain.

Each of these phases consists of several stages.

A. PHASE 1 - Recording administrative documents in the Ethereum blockchain

The execution of Phase 1 (Fig. 2) requires the payment of gas fees.

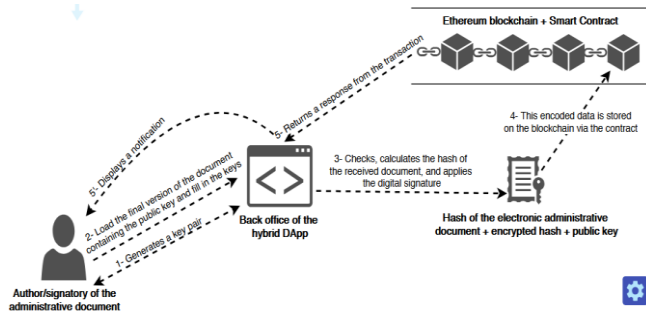


Fig. 2. PHASE 1 - Recording administrative documents in the Ethereum blockchain.

The process for this phase 1 is described as follows:

- 1- The author generates a pair of asymmetric keys through the Decentralized Application (DApp) back office. These keys must not be lost, and the private key must be carefully stored.
- 2- The author integrates their public key into the document so that it is visible. However, this remains optional. They then upload (via the DApp back office) the final PDF or Word version of the document containing their public key. During the upload, they enter the key pair. The front end transmits this data to the DApp back end.
- 3- The backend extracts the (text) content of the uploaded file and calculates a hash (or digital fingerprint) of this content. The backend then encrypts the calculated hash with the author's private key. This produces a signed hash, which is the document's digital signature. This data (the raw hash, the signed hash, and the author's public key) is encoded and sent back to the DApp frontend.
- 4- The frontend then transfers the encoded data received from the backend (raw hash, signed hash, and public key) to the smart contract – by calling the `storeAdministrativeDocument()` method–, which stores it on the Ethereum blockchain. During storage,

the timestamp is calculated and associated with the data by the smart contract. Thus, when a user wants to verify authenticity, the corresponding public key is used to decrypt and verify the document's signature.

- 5- A notification from the blockchain transaction is returned to the frontend. (5') This notification is customized and displayed to the user.

B. PHASE 2 - Verification or authentication of administrative documents on the Ethereum blockchain

This phase is illustrated in Fig. 3. No fees are payable here as it is simply a matter of reading data in Ethereum. This seems to justify *Jean-Paul Delahaye's* [11] expression "(...) freely and without charge, everyone can read (...)" in his definition of blockchain.

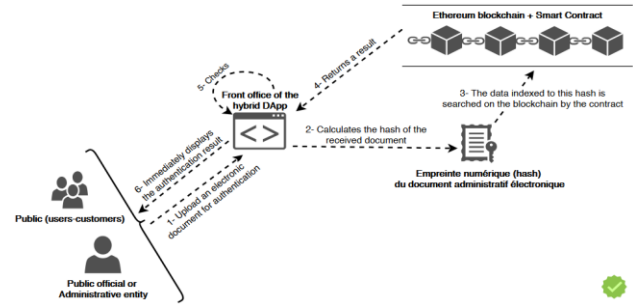


Fig. 3. PHASE 2 - Verification or authentication of administrative documents on the Ethereum blockchain.

Phase 2 also consists of several steps that can be described as follows:

- 1- The user (public or public official) uploads the PDF or Word document through the DApp front office whose authenticity they wish to verify. The frontend transmits this data to the DApp backend.
- 2- The backend extracts the (text) content of the uploaded file for authentication and calculates the hash of this content. This digital fingerprint (hash) is encoded and sent back to the frontend of the DApp.
- 3- The frontend uses this hash and queries the Ethereum blockchain via the smart contract (deployed there) to retrieve the raw digital fingerprint, digital signature, and public key previously stored in the blockchain (through phase 1). This search is possible by using the `getAdministrativeDocument()` method of the contract.
- 4- The blockchain returns a response to the DApp via the smart contract. This response is then transmitted to the DApp's backend for verification.
- 5- The backend decodes the data received from Ethereum and performs checks: if an occurrence is found (potentially intact document), the DApp verifies the digital signature associated with this occurrence via the previously stored public key. If this verification fails, then the signature does not come from the correct signatory, i.e., the author who validated the final version of the document. This is the verification of the document's authenticity. If the signature is valid, then the document is declared authenticated and authentic.

- 6- A notification is sent to the frontend. This means that the authentication result is immediately personalized and displayed to the user.

C. Implementation architecture

Integrating smart contracts into a traditional application allows interaction with the blockchain. For successful integration, we have adopted a clear DApp architecture (Fig. 4) where the responsibilities of each component are well defined.

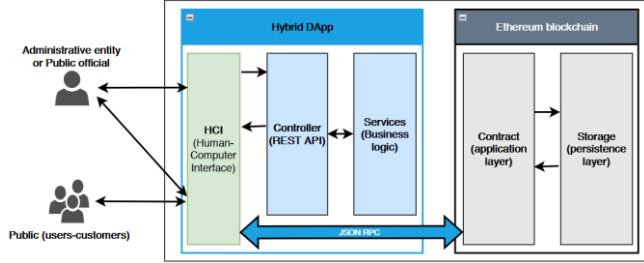


Fig. 4. Simplified layered architecture of the solution.

The Human Computer Interfaces (HCI or frontend) layer consists of two areas: a back office that is only accessible by the administrative entity, and a front office intended for the general public. The Controller layer, which is the upper web layer of the backend, is responsible for exposing REST APIs for the benefit of the HCI layer. These APIs provide resources (hash, signature, etc.) that are calculated by the Services layer (responsible for business logic). The responsibility of the application layer of the Ethereum blockchain is to host our smart contract and to work with the other layers of this blockchain to execute the contract. This application layer communicates with the HCI layer. It also communicates with the Ethereum persistence layer, where the hashed, encrypted, and encoded data from administrative documents is stored and searched. This is what enables decentralized storage of these documents through immutable transaction requests. The various interactions (transaction requests) between the EOA account and our smart contract are shown in Fig. 5.

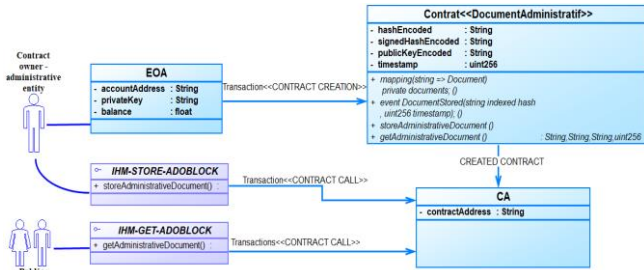


Fig. 5. Diagram of interactions between the smart contract and accounts.

IV. TESTING THE SOLUTION

A. Tools

The implementation requires at least one computer with a Core i7 processor, 16 GB of RAM, and 128 GB of SSD storage, with no operating system requirements.

The backend and frontend parts of the DApp were implemented with Visual Studio Code 1.96.2 using the Spring-boot 3.4.1 (Maven 3.8.1 + Java 17) and Angular 15 (HTML5, TypeScript/JavaScript, SCSS/CSS3) frameworks.

The smart contract was implemented and tested in **Solidity** ^0.8.28 on Remix-Ethereum IDE. The **MetaMask** v2.132.0

cryptocurrency wallet was integrated for blockchain transaction requests and signatures, which are made via the JSON Remote Procedure Call (**JSON RPC**) protocol to Angular via the Application Binary Interface (**ABI**). In terms of core technologies, we experimented with the **Hardhat** ^2.24.0 framework and the **Ethers.js** ^6.14.0 library to configure the test blockchain network (Localhost network), compile, and deploy our contract. In order to generate ECDSA (Elliptic Curve Digital Signature Algorithm) **cryptographic key pairs**, we use the Maven **bouncycastle** dependency; this allows us to use the **secp256r1** (P-256) elliptic curve for digital signatures of digital fingerprints calculated using the **SHA-256** algorithm. P-256 is compatible with Ethereum and the standards of the National Institute of Standards and Technology (NIST) [12] [13].

B. Simulations

According to our experiments on official announcements, storing a file of approximately 500 KB requires an average of 322,200 Gas, or 389,212.119378 Gwei (0.000389212119378 ETH, given that 1 ETH = 10^9 Gwei = 10^{18} Wei). These operations are instantaneous on the local test network.

The solution we propose, under the name ADOBLOCK, is accessible through a web browser. Figures 6, 7, and 8 show the main user interfaces.

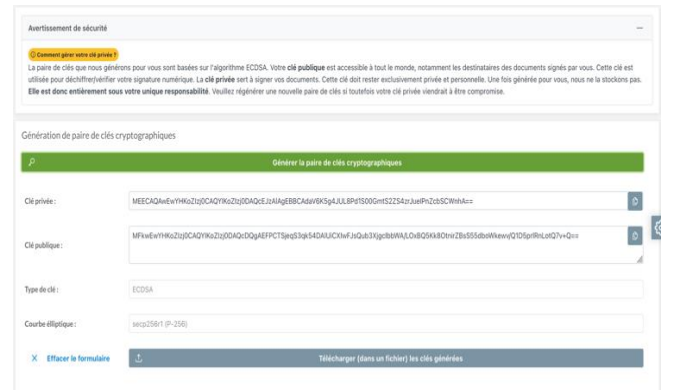


Fig. 6. Get private/public keys.

This interface in Fig. 6 allows you to obtain key pairs and save them in a compressed file. These keys are reusable as long as the private key does not appear to be compromised.

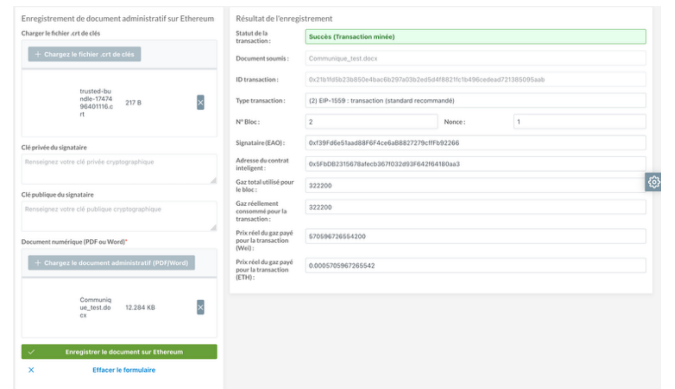


Fig. 7. Store administrative documents.

Fig. 7 shows the recording of an administrative document in the blockchain. In addition to the transaction status and hash, nonce, etc., the solution displays the amounts of gas used and fees paid.

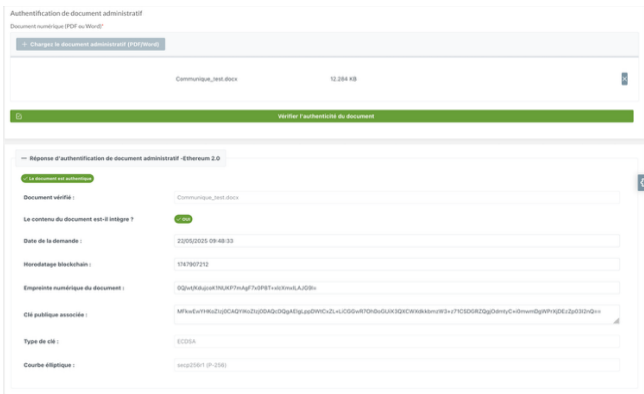


Fig. 8. Verify or authenticate document.

This interface displays the result of an authentication operation. It includes the timestamp, the integrity status of the document submitted for authentication, and the public key of the document's author found on the blockchain. The decoded digital fingerprint, key types, and signature curve are also displayed.

CONCLUSION

Faced with the rise of document falsification, it is crucial to leverage technological advances and innovations to rethink methods of authenticating and securing digital documents. To this end, we have proposed an approach to document authentication using blockchain technology. To test this approach, we have implemented a hybrid decentralized application that interacts with the Ethereum blockchain using a smart contract. In this solution, we have integrated the use of electronic timestamping, digital signatures based on asymmetric keys, and hashing techniques using the SHA-256 algorithm.

The results of our experiments confirm that blockchain technology shows promise for guaranteeing the authenticity and security of administrative documents. However, further consideration should be given to the sovereignty of digital data and the integration of other pipelines to support electronic documents with non-textual content.

REFERENCES